

Cognition

Have you ever stopped to think about some seemingly simple interaction, wondering how people do it? For example, consider Figure 5.1. What should the user press to be able to edit a photo? This can be solved by recalling from memory: “Hmm, does PowerPoint have image editing capabilities?” It can also be solved by means of visual attention, that is, by looking around for recognizable icons. Some reasoning may be required, for example, considering whether PowerPoint allows for importing the image type that is being used. In the end, the user may need to decide between trying to use PowerPoint, trying some other means, and giving up. Such thoughts and processing occurring in one’s mind are jointly called *cognition*.

The term *cognition* comes from the Latin word “cognoscere,” which means to know or learn. As a scientific term, it refers to mental activities that enable thinking and understanding. It is often taken to refer to how people process information in their mind; that is, how they perceive, learn, remember, reason about, and utilize knowledge. This view is called the *information processing view* of cognition. The study of cognition is a multidisciplinary effort spanning psychology, cognitive science, and neuroscience.

The main function of cognition in human–computer interaction (HCI) is to help the user control a computer to do what they want. Attention is needed to find elements, memory to learn and recall their properties, linguistic abilities to understand text and generate command names, and reasoning to make use of the knowledge obtained so far and deduce what happens inside the computer. Unfortunately, these cognitive capabilities have limits. People differ greatly in their cognitive abilities. Moreover, the way these abilities are used is affected by the type of task and the design of the user interface. Cognitive capabilities are used adaptively. Users learn and try to discover new possibilities and strategies; for example, they change behavior when a practice does not work. These properties of cognition make it important but also challenging to understand. There are five *elementary cognitive capabilities* that relate to our ability to interact with computers:

Cognitive control: Adaptively deciding goals (what to do), allocating cognitive resources to tasks, and changing the course of action as needed. For example, cognitive control is needed when a user decides to multitask by looking at their phone while driving [725].

Memory: Forming, maintaining, and accessing beliefs about objects that are not directly perceivable. For example, long-term memory (LTM) helps users locate previously seen icons more quickly [393].

Attention: Selectively processing part of the perceptual field, for example, by deploying visual attention on a screen or by sensing a tactile display by sweeping fingertips on it. We discuss visual attention in Chapter 3.

Reasoning: Applying transformation rules to beliefs to form *new* beliefs. For example, reasoning is needed to infer if software can do certain things when one cannot recall it doing that before [671].

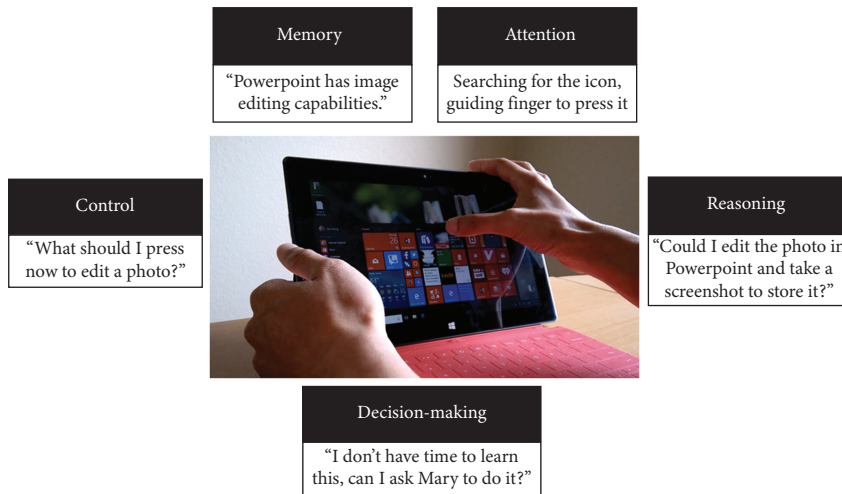


Figure 5.1 To use a computer, users need to choose actions that get the computer to do what they want. This is challenging because the computer is a non-transparent system. That is, users cannot “see” what actually happens inside the computer. The role of cognition in interaction is to help us overcome this challenge. Cognition underpins our ability to control actions, reason about the computer, learn from experience, and make decisions on what to do.

Decision-making: Interaction often requires decision-making. For example, a user might decide to install application A and not application B because A appears to have a more favorable cost and better features.

In this chapter, we discuss general findings about cognition that are relevant to HCI. This is followed by an explanation of how cognitive control allows us to direct our thinking and actions toward achieving goals. Then, we review working memory (WM), LTM, and mechanisms that assist us in learning. This leads to an examination of processes that allow us to reason and make decisions. We conclude by discussing how we can create cognitive models that allow us to simulate aspects of cognition to inform design.

5.1 General findings about cognition

Many researchers have investigated cognition. We summarize some general findings that are relevant to HCI.

Cognition helps us set goals and maintain focus on them.

Cognition helps users achieve their goals. A *goal* is some desirable state of affairs. In HCI, a goal often refers to something that people would like the computer to do for them. For example, a user’s goal could be to email a message to someone or to change some privacy settings.

Goals affect how cognition processes information. Human memory is less about storing experienced events veridically (accurately) and more about offering access to memories that may be useful for a given task at hand. Goals set expectations about how the world is structured and what might happen next.

An important phenomenon related to visual attention is *inattention blindness*. If users are given a goal related to one part of a user interface, they recall features related to that part and largely ignore the other parts. This holds even if the users did look at those other parts, as confirmed by, for example, eye-tracking data [275]. Later, when the users are asked about parts unrelated to the goal, they cannot recall the information contained in those parts.

Cognition is limited.

Human cognition is limited. For example, our WM is limited; we can only keep a few mental representations active in our minds. The typical WM capacity that can be simultaneously maintained active in our mind is thought to be two to four items [607]. Forgetting occurs in LTM. We cannot remember everything we have experienced, and as a result, we forget details of things we have attended. Visual attention is also limited. For instance, people can extract more information from the foveal region than the periphery (Figure 3.1). Finally, our capacity for abstract reasoning and planning is limited. We often resort to external aids, such as calculators and notes, to help us go beyond the limits posed by our cognition.

These limits are real and potent, although we may not be consciously aware of them. In computer use, these limits affect our ability to find items on display, make sense of the information shown to us, navigate through the information space, and remember instructions on how to use something.

Cognition is based on internal models of reality.

Cognition supports our ability to reason about things that are not directly perceivable. This ability necessitates the construction of internal models of reality that can be used to formulate goals and plans. One example of such an internal model is a *metaphor*. In HCI, metaphors help users understand what to expect from a user interface. The desktop metaphor uses spatial concepts that are rooted in our everyday experiences in the physical world. For example, folders are *on*, not *under*, the desktop, and they can be moved *into* the trash when they are no longer required.

Cognition is necessary for learning and adaptation.

For some time, it was believed that human action is driven by *plans and reasoning*. That is, we plan sequences of actions and execute them. Suchman [804] challenged this view in studies of repair and maintenance personnel of Xerox photocopiers. Suchman [804] found that even in a highly regulated profession, a worker's action is not heavily scripted but requires constant adaptation and planning. While planning is carried out, such plans are often noncommittal sketches that need to be revised and updated.

The present understanding is that our cognitive, motor, and perceptual processes are constantly adapting—forming beliefs, trying new tactics, and fine-tuning. This is needed to respond to the structure of the environments we experience. This need is particularly pronounced in HCI, as both the systems people use and the ways people carry out work keep changing. This results in users having to continually formulate new plans and adapt to new ways of working with computer systems.

Hence, cognition is not simply passively processing information from external environments and reacting to it. Instead, cognition takes actions to facilitate its own functioning. We use external aids, such as notes, calculators, and browsers, to augment our abilities. Over time, such dependencies affect the way we use cognition in interaction. For example, since the uptake of the GUI, which relies heavily on visual recognition, we have less need to use our LTM to store computer commands that would be typed in a command prompt.

Cognition requires energy and effort.

Mental effort refers to the use of energy for controlling thinking to achieve our goals [236]. Mental effort has two main components. *Task effort* refers to the additional energy required in response to an increase in the computational demand. This can occur when we face a novel environment, for example, when learning to use a new user interface. *State effort* refers to the energy required to protect performance from physiological fatigue, which can be caused by sleep deprivation, among other factors.

Effort may sound like a negative concept in that it is something that limits performance. However, it also has a positive function: The feeling of effort prevents us from overconsuming energy through less important activities.

5.2 Cognitive control

Cognitive control refers to our ability to direct thinking and action toward achieving some goal. When using a computer, cognition must decide between possible actions to reach a goal. It must activate the right representations for achieving that goal [671]. It may also need to share representations among multiple tasks at the same time. In dynamic task environments, it needs to learn to control action in a predictive fashion.

The basic problem of cognitive control is the following. At any given time, countless stimuli bombard our senses, and we have several options for sharing our limited resources among them. How can we ensure that the *right options* are allocated the *right resources* at the *right time*? For example, just now, you could stop reading and start watching a movie. Similarly, if a notification pops up, do you immediately attend to it or do you first complete what you were doing? If you knew there was an important message coming, such as a letter of acceptance for a job application, you may want to check it immediately, which might be a rational decision. Cognitive control is critical to performing tasks in information-rich environments. The concept has been used to derive implications for the design of multi-part tasks (e.g., the use of automatic teller machines, or ATMs) and regulations for multitasking settings such as driving (e.g., suggesting minimum limits for maintaining sufficient attention on the road).

5.2.1 Activating goals

Cognitive control is also needed to activate the right subgoals at the right time. Faced with a complex task with multiple goals, cognition breaks it down into simpler, more manageable parts. Altmann and Trafton [16] explained this with a *goal activation model*. Contextual cues *prime* subgoals. For example, when we see a familiar intersection on the way home, it subconsciously activates our next subgoal. *Priming* is about the subconscious activation of concepts by perceived cues. However, previously activated goals can *interfere* with the activation of the relevant goal. Cognitive control is required to suppress the activation of interfering goals. The model can explain *postcompletion errors*, which result from the user forgetting to carry out an action that should be taken *after* achieving a goal. For example, one may forget to take the credit card out of an ATM after receiving cash unless the ATM issues a reminder to retrieve it.

5.2.2 Activating task representations

When a task is performed, cognition needs to make relevant task representations available. Cognitive control operates through two mechanisms: the *activation* of relevant representations in

memory and the *inhibition* of irrelevant representations. For example, consider the act of searching for an application icon on a desktop. Let us assume that this task is challenging and that you have already looked at several locations. Cognitive control is needed to select and plan the locations to visit next as well as to prevent you from revisiting locations you have already visited. The latter phenomenon is called *inhibition of return*. Without this ability, you would be constantly revisiting locations, which would make the search less efficient.

This kind of regulation is important in information-rich environments. Inhibitory control is needed to avoid attending to every flash or blink on an interface and maintain focus on the task at hand. In this way, cognitive control helps us balance between an internal and external locus of control. Without internal control, we would be at the mercy of our environment. Any pop-up or notification would capture our attention, and we would not be able to complete our tasks.

5.2.3 Choosing actions

Cognition is necessary to choose what to do. A central problem in computer use is that *rewards are delayed*. In other words, we cannot immediately obtain what we are interested in. Instead, we must choose one action now, then another, and so on. Cognition helps us solve sequential decision-making problems. However, how do we know which action to choose at a given moment when the goal is not immediately available but instead distal? An essential ability for effective interaction is to evaluate actions by considering their long-term rewards.

Two key mechanisms underpinning decision-making are posited by cognitive sciences [719]. First, through experience, users learn to associate an action with *recurring contextual cues*. For example, the familiar wallpaper of your mobile device is associated with certain action possibilities, such as pressing on some icons. How are such mappings established? Trying things allows users to learn *value estimates* of actions available in a particular state. The associative learning of such value estimates occurs subconsciously over time. It does not require effortful cognitive control; it is largely automatic. For example, you may notice that when you start using a new mobile device or operating system, it is hard to locate things. After a while, you start to find the items that you use frequently more quickly. However, this type of associative learning is slow to start and slow to update. If the user interface changes, users need to relearn some associations.

The other mechanism is *reasoning about and simulating possibilities in the mind*. For example, before selecting what to do, we may think about possible routes in a virtual environment. This requires cognitive control; you need to activate representations and compare your options, which requires effort.

5.2.4 Multitasking

We often need to carry out multiple tasks at the same time. Consider, for instance, driving a car. At any given time, there are numerous things you *could* do. You could be texting, attending to the road, talking to your passengers, checking the gasoline level, changing the AC settings, and so on. Cognitive control is crucial for effective *multitasking*. The problem for cognitive control in multitasking is how to allocate limited resources among the tasks we want to perform simultaneously.

Multitasking is a resource-sharing problem. The resources include those defined by attention, the motor system, and WM. Some of these resources can only be shared in an all-or-nothing fashion, whereas others can be shared in a graded manner. For example, you only have one dominant hand; in most cases, it cannot be shared in a graded manner between two tasks. However, in some cases, different tasks may be completed by different fingers. Visual attention, likewise, can

only be focused on a single location in space at a time. Conversely, auditory attention can be somewhat better shared among sound sources around us, even when the sources are spatially apart.

The *multiple resource theory* (MRT) of Wickens [888] provides a rough but useful heuristic for understanding how resource conflicts arise in multitasking. The MRT cube is presented in Figure 5.2. The cube can be used to estimate whether two tasks, when carried out simultaneously, will cause conflicts in resource utilization and should therefore not be performed simultaneously. For example, consider two tasks, A and B:

- Task A is playing a video game on a console;
- Task B is following a conversation on social media via a mobile device.

The MRT cube in Figure 5.2 consists of segments divided into three axes. These segments denote limited-capacity cognitive resources. To apply MRT, you mark all the segments that the to-be-done tasks need. If two or more tasks end up occupying the same segment, conflict will emerge and negatively affect performance.

Returning to our case:

1. Both A and B rely on the visual modality, so there is resource competition in this aspect.
2. Both A and B rely on manual responding, leading to resource competition.
3. While A relies on spatial processing, B relies more on verbal processing, so there is no competition here.
4. Both A and B can involve ambient auditory and visual stimuli (e.g., notifications, sounds).

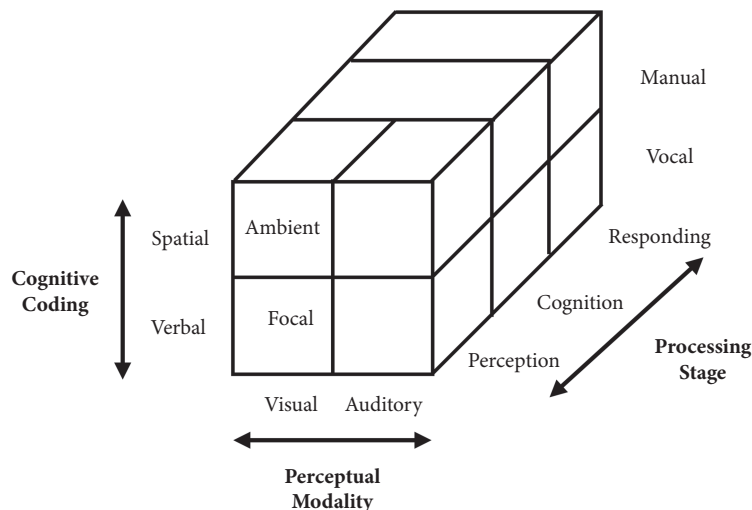


Figure 5.2 Multiple resource theory explains why certain task combinations are more prone to conflicts when multitasking than others [888]. The cube has three axes that help us understand the motor, perceptual, and cognitive demands of a task. To assess whether two tasks can lead to resource competition, you start by marking the slots in the cube that the tasks occupy. If the two tasks occupy the same slots, we can predict that there will be resource competition and thus performance degradation. Different types of resource competition occur in different cells.

MRT predicts that Tasks A and B cannot be carried out simultaneously unless Task B relies on notifications. Task A can only be carried out with visual attention, and so can Task B. This means that at any given time, the cognitive controller must decide which task gets attention. Hence, attention must be switched between the tasks. However, the tasks may not need to be processed simultaneously, as is the case with asynchronous conversation.

If notification sounds are turned on, visual attention does not need to be sustained on the social media app all the time. When a notification pops up, it must be decided whether to disconnect the video game or not. Attention can be shared momentarily to read the message while keeping the hands on the video game controller.

What happens if two tasks require the same cognitive resource at the same time? This scenario necessitates cognitive control to decide which task receives the cognitive resource at a given time. The result is interleaving: Switch to Task A for a few seconds, then switch to Task B, and so on.

This kind of interleaving is adaptive. In driving, for example, we do not switch between tasks arbitrarily. Doing so would result in a dramatic increase in accidents.

Since every task switch is somewhat costly, as you lose a few hundred milliseconds just for the switch and you may forget part of the task if you switch too abruptly, the cognitive controller must be sensitive to both the estimated gains of switching to other tasks and the benefits of carrying out the current task. For example, switching too late may result in a missed opportunity in another task. Switching to another task for too long may also incur excessive costs. This is because the longer a user is prevented from working on a task, the longer it typically takes to resume that task later on.

Empirical studies have investigated people's ability to switch between tasks and learned that our task-switching strategies are sensitive to switching costs [384]. Cognition seeks to perform switches at natural breakpoints. In other words, users prefer to avoid switching immediately and instead postpone interruptions to natural boundaries between subtasks.

Uncertainty is a prevalent characteristic in almost all multitasking environments. Many tasks we need to complete in everyday life are dynamic. This means that they change on their own, even when we do not attend to them.

One implication is that at any given time we are not attending to them, we are uncertain about their state. Furthermore, this uncertainty accumulates: The longer you leave a task unattended, the higher the uncertainty about its state.

For example, consider someone driving a car. The longer the driver attends to their smart-phone, the more uncertainty the driver will have about the driving environment. Guidelines for safe driving recommend a maximum of 2.0-second shifts in attention away from the road [612].

To decide when to switch to a different task, cognitive control needs access to sufficient information to maintain estimates of subtask states and their uncertainties. It needs to track what it does *not* know about the world. When the risk of not attending to a specific task becomes too high, the cognitive controller needs to move attention to this task.

For example, when driving in the rain, glances to secondary tasks tend to be shorter than when driving on a day with better road visibility. This is because there is more uncertainty about the driving environment when visibility is poor. When cognition does not have access to sufficiently accurate estimates of the costs and rewards of a task, it may incorrectly engage with an irrelevant task.

5.2.5 Predictive control

When the task environment is dynamic, that is, changing on its own regardless of the user's actions, cognitive control needs to be *proactive*. Instead of waiting to be triggered by the right stimulus, it needs to anticipate the need for a representation even before the event occurs.

To enable this, the cognitive controller requires an *internal model* of the world. The internal model should be able to predict what *might* happen if an action were taken. However, this internal model should not determine actions, as perception is also an important determinant for choosing the correct action. Cognitive control needs to integrate all of these aspects.

One hypothesis about how we do this is called the *Bayesian brain* [424]. According to this hypothesis, the brain is a statistical estimator that assigns probabilities to hypotheses about the state of the world. As the brain receives new sensory information, it updates the probabilities of these hypotheses. This is referred to as updating the beliefs. This updating follows the rules of probability theory, including the well-known Bayes' theorem. A Bayesian brain uses past experiences to anticipate new experiences. The less expected a new experience is, the greater the surprise. A Bayesian brain attempts to optimize its knowledge about the world to minimize such uncertainty.

For example, as you interact with an AI-driven feature such as autocomplete, you may think: "What would happen if I did this, how would the feature react?" We become better at making such predictions thanks to belief updates. Even in surprising new situations, such as when typing a word borrowed from another language, we utilize such beliefs to predict how the AI might react—would it know how to complete the word?

Consider a collaborative puzzle game. Your teammate is running very fast from one side of the display to the other. How do you know when to trigger an action to throw a piece to her? A Bayes optimal perceptual system does not represent the property of an object, for example, the velocity of an object moving on the display, as a single variable. Instead, it is a conditional probability density function $p(V|I)$, where I is the available sensory information (e.g., the change in the object's location since the last observation). This specifies the conditional probability that the object is perceived to be moving at different speeds V based on the available sensory information I .

In other words, the probability that a user correctly interprets a certain velocity depends on what is shown on the display. However, an optimal observer must also take into account the relative uncertainty of each source when computing an integrated estimate. When one cue is more biased or unreliable than others, it should be given proportionately less weight. For example, when gamers estimate when to press a button to intercept an object, they give different weights to visual information, auditory information, and their own estimates of the object's movement [468]. By integrating all the evidence, they get a more reliable estimate than by considering either piece of evidence alone.

5.2.6 Cognitive workload

Maintaining cognitive control is tiresome. Do you remember the first time you learned how to program or how to use a spreadsheet application? Do you recall how taxing it felt to carry out a simple task, such as keeping track of a mouse cursor and remembering what to do, and when? Having to continuously decide what to do is mentally demanding, and doing this for an extended period of time can cause fatigue or stress. In addition to having to decide what to do, users also frequently act under uncertainty and some level of risk, both of which further increase stress.

The experience that follows from such effort and stress is called *cognitive workload*. The perceived workload can be measured with the NASA Task Load Index (NASA-TLX) questionnaire (Chapter 7). Over time, with experience, people develop *minimum effort task strategies*—routines that diminish the perceived workload. In practice, this often means requiring less executive control. This requirement also decreases with *automaticity*; that is, when users learn a skill, it

consolidates and becomes more automatic. This allows users to build on previously learned skills when refining some skills or acquiring new ones.

5.3 Memory and learning

Memory is critical in computer use. Without memory, you would struggle to complete even simple tasks, such as filling out a registration form, changing settings, or navigating a page. We need memory to keep track of things, reason, recall, and recognize. However, memory does not refer only to consciously accessible memories. According to dual process memory models, there are separate neural processes for conscious and unconscious remembering [903]. What one has experienced can affect performance in an implicit way. For example, you may “just know,” without explicitly recalling, where certain common commands are, such as where File is in an application menu.

Current neuroscience evidence suggests that several distinct memory systems are involved in interaction, each with different functions, purposes, and neural bases [789]. It is important to understand these systems because, in almost every interactive task, we have more than one at play. This also challenges a common lay conception of human memory. Memory is *not* like a storage house or a hard disk that lets you store items and retrieve them later.

5.3.1 Working memory

Working memory refers to the temporary maintenance and manipulation of representations in the mind that are needed for action [607]. WM is set apart from LTM due to its time- and capacity-limited nature. We can only maintain a few items at a time in our WM. The nature of this capacity has been the subject of several decades of research.

Early research suggested that our WM is limited to only two to six items at a time. This was initially known as Miller’s magical number 7 ± 2 and was later revised to a more conservative 4 ± 2 . According to this view, at any given time, we can only hold about two to six items in WM; trying to hold more items will result in losing some of the existing ones for two reasons. First, this would exceed the capacity limit of WM; second, the items would not be rehearsed. Active rehearsal is needed to keep items in WM. This is what makes the use of WM seem mentally taxing. Access to the items fades quickly unless it is actively maintained.

The limited capacity of WM can be demonstrated and measured with a simple task called the *n-back task*: Ask your friend to read aloud random numbers between one and nine at a constant pace. Your task is to report the *n*th item *before* the last one. To be able to do this, you must now keep active *n* items in your WM and update the memory contents upon hearing a new number. The task is much harder than it sounds and exposes the radically limited ability we have for the simultaneous maintenance of multiple ideas. It is best learned by trying it out.

Currently, it is thought that the capacity of WM is even smaller than what Miller estimated, perhaps just three to five items in young adults [170]. Moreover, WM is argued to not consist of memory “slots,” and its limits emerge due to the inability to maintain several active representations in associative memory networks. In other words, items in WM are not fully “lost” when we stop rehearsing them; they just lose their active state and it becomes increasingly harder to reactivate them.

WM is needed in many interactive tasks. Consider, for example, the task of copying text from one application to another. You need to not only remember what to copy but also keep track of where you are in the to-be-copied text. Another example is interruptions. We get notifications and

switch between tasks all the time. The more disruptive the interruption, the more we rely on WM at the time of the interruption. If there is a single takeaway for design from this research, it is to avoid relying on WM. This can be done in cases where representations can be encoded into the LTM.

5.3.2 Long-term memory

Fortunately, users do not have only WM to rely on. LTM refers to memory systems that are responsible for exploiting past experiences. Figure 5.3 shows a taxonomy of LTM systems. Evidence of these systems comes from brain imaging and studies of patients with abnormal brain functioning.

Declarative memory refers to long-term memories that can be consciously experienced, or *explicit memory*. By contrast, *non-declarative memory* refers to *implicit memories* that affect our behavior without conscious recollection.

For example, how do you remember your password? This is a good example of how both declarative and non-declarative memories contribute to our ability to perform interactive tasks. For one, you can recall a password by recalling the act of typing it—a memory of moving your index finger on a keypad to enter the PIN one character at a time. You just know this. However, you might also be able to recall the password as a word. When users are asked to *generate* a password, they are asked to think about its letters and words. Thus, they may remember the password using semantic associations. However, for a password that we have entered several times, we may have forgotten the semantic representation of the password and rely on our ability to “just type it.” For example, you may have noticed that it is difficult to type a familiar password using an unfamiliar keyboard layout.

The declarative memory system contains the *semantic* and *episodic* subtypes. Semantic memory is responsible for propositional knowledge, for example:

- Folders contain documents.
- “Save” is related to “Open.”

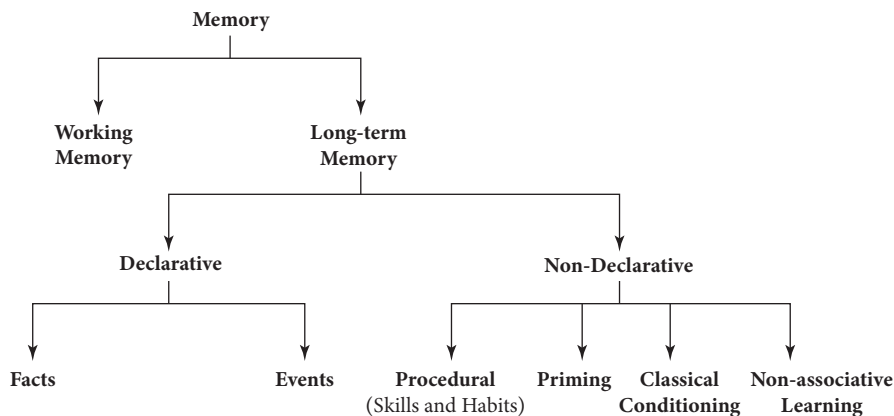


Figure 5.3 Taxonomy of human memory systems [789].

Episodic memory can be thought of as mental time travel: It allows us to re-experience past events. “What did I do last time I wanted to print in A3 size?” Programmers who scroll program code keep track of where they are using episodic memory.

Non-declarative memory systems are further divided into procedural, priming, conditioning, and non-associative learning. *Non-associative learning* refers to reflexes, such as retracting the hand from a hot stove. *Procedural memory* refers to learned sequences of actions and thoughts. Procedures are like recipes. Instead of controlling actions one by one, the user can trigger them as a sequence. After extensive repetition, actions become associated to a larger extent, which allows the user to execute them without conscious awareness. For example, you may “automatically” know the sequence in which you enter your name and password to log in.

Priming refers to the unconscious effect of previously seen stimuli on responses to subsequent stimuli. Priming has an important role in preparing us to respond in a manner appropriate to the given context. Contextual cues “prime” or make us more ready to give certain responses. For example, asking a user to press the left arrow button in a gaming context may lead to faster responses from gamers than from users with a form-filling background.

Conditioning refers to the learning of actions that are triggered (conditioned) by the environment. While only the readiness to act is affected in priming, in conditioning, the probability of action selection changes. Conditioning is an unconscious association between the environment (the cue) and the response. The strength of this association can be affected by reinforcement. *Positive reinforcement* is a reward given for successful behavior; *negative reinforcement* is a penalty or punishment for unwanted behavior. When reinforcement is removed, a specific behavior can be inhibited and eventually removed. Conditioning is one mechanism that can be used for behavior change (Chapter 6). A positive behavior, such as ceasing smoking or starting physical exercise, can be reinforced by introducing a positive reward. In gaming, a positive reward can be, for example, a loot box or a badge.

The multiplicity of memory systems provides desirable redundancy that improves robustness: If one system fails, the other can be used. Vice versa, overly relying on a single system, while beneficial when circumstances stay the same, can be detrimental to our flexibility to adapt and recover from errors.

5.3.3 A three-stage view of memory functioning

Time is a defining aspect of interaction with computers. Most things we do with computers take more than a few seconds; some activities take years. It is therefore natural to look at memory functioning from the perspective of time. Both declarative and non-declarative LTM can be understood in terms of three *stages* organized in time:

Encoding: Memory traces are formed during interaction.

Storage: The traces are retained between encoding and retrieval; some are forgotten.

Retrieval: The traces are retrieved at a later stage, for example, when using the same user interface again.

Encoding affects what we can, in principle, remember later on. The type of encoding depends on how much attention is paid and how deeply we process information. The *levels of processing (LOP) effect* refers to the association between the depth of mental operations carried out during the task and the strength of the encoded memory trace [172]. The deeper you process something

that happens on the interface, the better you will remember it. Items on a user interface that are interesting are typically processed more deeply. Those that are just glanced at and rejected are encoded in a shallow way and are more quickly forgotten. For example, when some users were asked to navigate or read the content on a web page, they remembered the corresponding elements well and almost entirely forgot the others [624]. An exciting application of the LOP effect in gesture-based interaction is presented in Paper Example 5.3.1.

To retrieve a memory trace, we need some *cue*. The cue can be something you see, feel, or do. It can (or can fail to) reactivate the original trace. Depending on the type of cue available, three types of recall can occur. First, a *free recall* refers to an attempt to retrieve a trace by self-generating cues; that is, without relying on externally presented cues. For example, if you are deciding which programs can be used to complete some task, you typically do this without having those programs visible. Second, a *cued recall* refers to trace retrieval via external cues and is generally much easier than a free recall. Success, however, depends on the cues. Third, a *recognition* is an extreme version of a cued recall where the whole object is presented instead of its part. For example, when the whole app logo is presented, the user simply needs to recognize it as the one that is being searched for. You need to decide whether it corresponds to a particular class or experience you have seen before.

Our visual recognition memory is amazingly good. You can recognize hundreds of faces you have seen during a football match, or many passwords you have used during your life. However, recalling them would be very difficult. This ability is exploited by *graphical user interfaces* (GUIs; Chapter 28). When we need to recall a command, such as in *command language interfaces*, we need to use free recalling (Chapter 27). This is effortful. By contrast, with GUIs, we can rely on strong visual cues and simply recognize familiar graphical elements.

There is an interesting interplay between cues and recalls, and interface design affects this interplay. *Encoding–retrieval symmetry* refers to the similarity between the conditions in the encoding of an item and those in retrieving it. High symmetry facilitates retrieval. For example, it is harder to retrieve a password if the color of the login screen has changed.

Paper Example 5.3.1: What makes a gesture memorable?

Gesture-based input holds great promise. However, memory is a limiting factor in gesture-based interaction. Unless gestures are demonstrated to users, users must actively recall them in order to use them.

Nacenta et al. [574] wanted to understand what makes a gesture memorable. They compared three types of gestures. The first type comprised gestures specifically designed by designers for a particular application. The designers were informed about the goal of producing memorable gestures that would also be easy to perform and easily recognizable by the gesture recognizer. The second type was stock gestures, that is, generic gestures that would typically be preloaded in an application. The third type was user-defined gestures. Here, the researchers asked users to generate their own gestures.

These types of gestures were examined in three experiments. Overall, it was found that user-defined gestures result in better recall than the other types of gestures. User-defined gestures were recalled up to 44% better than others. They were also perceived to be less effortful and time-consuming. Users also rated them higher in terms of gesture preference. The authors recommend allowing for user-defined gestures when possible.

5.3.4 Forgetting

What happens between retrieval and encoding? Memories are lost. How does this happen?

According to *decay theory*, memory traces lose their activation or strength over time. Events in the recent past usually have more pronounced importance for actions in the present. The more time has elapsed since a memory was activated, the less probable it is that it can be retrieved. According to *interference theory*, memory traces get confused with each other. When we try to retrieve a memory trace, another (false) memory is activated at a similar level. Memories are not simply forgotten; they become mixed up.

Moreover, not all memories are forgotten in the same way. Our encoding and forgetting processes adapt to the statistical structures of our environments. If all previous memories were competing for attention, we would have difficulties retrieving what we need. Nevertheless, we need to forget in a way that keeps the organismically most important memories available.

With practice, computer users become better at predicting what they will need to remember later. As we discuss in Chapter 21, memory adapts to the statistical distribution of the environment. For example, to recall a password, we need to retrieve it from our LTM—but what determines how easily it can be retrieved? Ecological theory suggests that human LTM evolved to help survival by anticipating organismically important events. It is evolutionarily important to remember things that are important for survival and thriving. The expected value of remembering it in the future should gain priority in how deeply something is encoded in memory. Paper Example 5.3.2 shows an example of predicting how people forget where things are in a UI.

Paper Example 5.3.2: Modeling how users remember where things are on a user interface

Theories of human LTM can explain how we remember, and why we fail to remember, interface elements over time. Consider, for example, the icons on your smartphone. Over time, you learn to find them with increasing performance without putting conscious effort into it. How is this possible?

LTM is an associative network in which nodes that encode UI elements are associated with other nodes [22]. An item in memory has a base level of activation that determines how likely it is to be recalled. It also has associations with other items, which can help or inhibit its retrieval. These associations have different strengths.

Consider a user who occasionally taps icons. When an icon is selected, its association with its properties, such as its color and location, increases. Afterward, the association starts to decay. The farther in the past the previous accesses are, the less they contribute to the activation, accelerating the decay of the memory. This can be expressed mathematically. The base activation of the icon B_i is

$$B_i = \ln\left(\sum_{j=1}^n t_j^{-d}\right), \quad (5.1)$$

where t_j is the time since the j th visitation of i , and d is a decay parameter.

When retrieving the location of an item, the probability of its retrieval is related to this base level. If $B_i > \tau$, where τ is some minimum threshold for retrievability, the item is retrievable. If a cue related to the item is given, the *source activation* B_{sa} can be added to B_i . This is called

continued

Paper Example 5.3.2: Modeling how users remember where things are on a user interface (*continued*)

spreading activation, that is, the activation of whatever is in the focus of attention boosts the activation of relevant items, thereby facilitating their retrieval.

Finally, the time to retrieve the location of the item, T_i , depends on its activation. This is given by

$$T_i = Fe^{-fB_i}, \quad (5.2)$$

where F and f are individual-specific constants.

5.3.5 Learning over time

Every time you do something, subtle neural changes take place. These changes prepare you so that you can better tackle the task in the future when you encounter it again. Over time, the change in performance is dramatic. What used to be hard and taxing becomes more automatic, and therefore more effortless. However, *how* you practice, as well as whether you deliberately practice the task or simply execute it, affect how well you learn.

Perhaps the most dramatic improvements in performance occur early on. For example, after just a few tests with a new input device, we typically see large changes, which then become proportionately smaller with more practice. Another way to say this is that increasing practice provides *diminishing returns*.

For example, in a motor learning task, this relationship is captured in the *power law of practice*:

$$RT = aP^{-b} + c, \quad (5.3)$$

where RT is the reaction time, P is the number of completed practices, and a , b , and c are free parameters whose values are case-specific. The law portrays a quantitative relationship between the best-effort performance and the amount of practice. It has been found to describe motor tasks, such as pointing or rolling cigars, and to some extent mental performance, such as retrieving facts or solving arithmetic tasks. The free parameters (a , b , and c) control different aspects of the slope: the intercept and the rate of change. An example is given in Figure 5.4. In the figure: Which curve shows faster learning? What do the three free parameters (a , b , and c) mean?

When performance improves, the expected reaction time is not the only factor that changes; the performance also becomes more *stable*. In other words, the variance in performance decreases. In some cases, this trend has been observed to follow the power law of practice, although with different free parameters. The point is that early on when learning a skill, we struggle to keep performance stable and see large differences across trials. Large improvements are seen during the first trials, and performance stabilizes further, albeit with diminishing returns, with more practice. Stability is important in HCI because it means the user can better predict the outcomes of interaction, is less prone to errors, and has greater self-confidence.

With extensive practice, skills transition from controlled to automatic. *Automaticity* refers to the fast and effortless performance of skills. For example, when first using the camera application on a smartphone, you probably struggled to find some settings and even basic functionalities such as

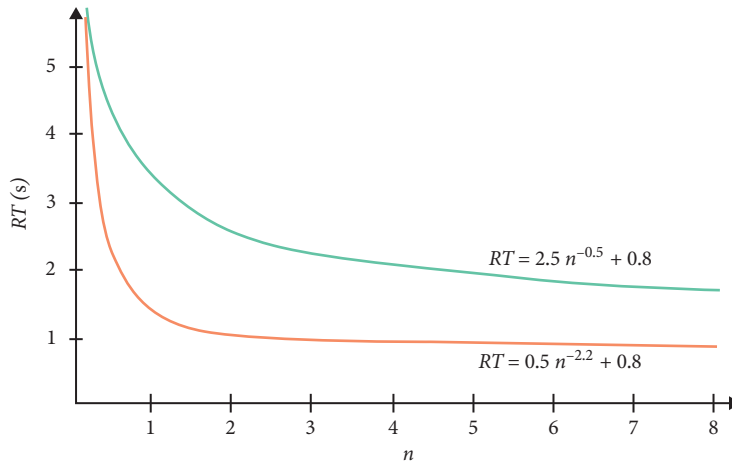


Figure 5.4 The power law of practice states a relationship between the reaction time and the number of practices: $RT = aP^{-b} + c$. Here, it is shown with two parameter sets.

video recording. However, with practice over time, you can now focus more on the object of photography than on the camera itself. The downside of automaticity is its inflexibility. Automaticity is often described as a ballistic skill: The skill is triggered by some cues and carried out to its completion with little opportunity to redirect it. This means that with increasing practice, users may become insensitive to changes in a design.

5.3.6 Designing practices

In many settings, we are not just using computers; we invest time and effort to practice their use. *How* we practice matters. The most inefficient way to learn a skill is to just mindlessly do it. Significant performance gains can be obtained by putting some thought into training.

The basic parameters of a practice regime are:

1. Scheduling of practice: When practice happens
2. Selection of practices: Which practices are done
3. Feedback: How feedback informs changes in strategy.

In the following, we discuss these one by one.

Scheduling practices: How practice trials are scheduled over time has a significant impact on learning. The worst schedule is *massed*:

| | | | |

Here, all practice takes place in a compressed timeframe, which leaves very little time for the consolidation of memories in the brain. Somewhat better is *equi-spaced* training:

| | | | |

Here, practices take place not too long after each other but not too shortly after either. The best training schedule is *expanding practice*, represented here with an exponent of 2:

This means that practice will take much longer. However, it provides much more time for slower learning processes in the brain.

Selection of practices: Perhaps the worst way to learn a skill is to just perform the activity. *Focused practice* refers to the selection of *isolated* practices with clear objectives and corrective feedback. These practices should not be arbitrary, but focus on those aspects that show room for improvement. But how to identify such practices? This can be done by a human mentor or by tracking the user's development and comparing it against the expected level of performance at that level of training.

Feedback: Feedback is important in HCI, as it helps users set goals and correct mistakes in computer use. Consider a computer game you have recently played: What kind of feedback does it provide about your performance? For example, it may provide a numerical score or indicate which level you have mastered. It could also be more specific; for example, it could tell you how many events of a particular kind you have mastered. Feedback can be of three different kinds: (1) performance-indicating, for example, a score or words per minute in text entry, (2) performance-correcting, providing qualitative feedback to help identify problematic areas of performance, and (3) strategy-enhancing.

5.3.7 Stages of skill development

How good do you think you are at using your mobile device? It is probably hard to tell. Skill development in HCI is often described in three stages: novice, intermediate, and expert.

The novice stage is characterized by a struggle to complete tasks and high variability in performance. Performance is neither high in comparison to peers nor stable. While performance improvements can be obtained through repetition for simple tasks, for complex tasks, this may not suffice. After the initial gains from repetition, the next improvements are achievable only through changes in the *interaction strategies*. Learning is very specific to the interaction strategy. For example, if you first use a menu for selecting commands and later shift to keyboard shortcuts, there is little—if any—skill transfer from the first skill to the new one. Any change in strategy may initially reverse previous advances in performance. Users may initially perform much worse than before the shift. However, performance will improve quickly, and in some cases according to the power law of practice. The power law of practice can be used to estimate whether the new strategy will eventually allow for achieving a superior level of performance.

Most of our computer skills are self-taught. Without deliberate practice, skill development stops at the intermediate level. This level is characterized by performance that is acceptable for most regular activities. However, the acquired skill may not be robust or generalize to new tasks. For example, in a study of mobile users, the skills obtained by intermediate-level users were found to be device-specific [627]. The users mainly learned their skills via early familiarization with a device, repeated use, and problem-solving situations. While experienced users exhibited superior performance in tasks, they failed to transfer what they had learned to *other* tasks due to a lack of deeper conceptual representations of how the technology works. Instead, their performance was attributed to their being overall better at navigating interfaces and knowing where things are.

The highest level of skill, the expert level, is only achievable with years of deliberate practice. Significant practice, in the order of thousands rather than hundreds of hours, is required. The design of such practice matters and is normally done by coaches and other experts, such as in

the case of video games. Practices should be isolated, focus on areas of low performance, and provide corrective feedback. Because such practice can require significant effort, the motivation for persisting is important: How to keep the user motivated, particularly when the practice is not fun?

In this regard, the role of coaches, mentors, and exemplary users is important. In *apprenticeship learning*, an expert user guides the practice. Consider, for example, a medical doctor teaching novice users how to use a new patient record system. An experienced user can be involved in the selection and scheduling of practices, the provision of feedback, and motivating others.

5.3.8 Knowledge-in-the-world, knowledge-in-the-head

Finally, users are not at the mercy of what they remember. They can flexibly *externalize* knowledge and use such externalized representations for recalling purposes. The key distinction is between *knowledge-in-the-world* and *knowledge-in-the-head* [288]. For example, to avoid disclosing our passwords to others, we would like to store and maintain them in our LTM, safe from peering eyes. This is knowledge-in-the-head. However, we often end up storing passwords on slips of paper, files on computers, and so on; in other words, knowledge-in-the-world. Our cognitive functioning may become dependent on having access to that externalized knowledge, negatively affecting knowledge-in-the-head.

People also actively manipulate the environment to change the cognitive requirements of a task. The case of Tetris players is an eye-opening one. In a study of expert Tetris players, it was found that not all of them try to mentally rotate a piece to fit it to the landscape below [417]. Instead, some players quickly flip the piece to *visually recognize* the best slot. This is beneficial because we are fast in recognizing solutions but much slower in mentally simulating events.

Distributed cognition can be very involved in complex task environments. Consider an airplane with a pilot and a co-pilot operating in the cockpit to land the plane [364]. There are numerous computer devices showing the state of the airplane. There are standard operating procedures—checklists that show what to do and in what order. Moreover, there are two pilots who communicate to establish common ground on the status of the airplane. Speech, such as telling the status of a checked meter, can provide a status update for the other pilot. Together, these devices, practices, and communications form a distributed cognition. The idea is that we cannot attribute cognition to a single brain, and its functioning is best understood by considering the joint operation of the pilots and the environment.

5.4 Reasoning and decision-making

We often face a situation in interaction where we do not have a direct solution to something that we need. We cannot see anything or recall anything that could help us proceed.

Reasoning is about thought processes that allow us to conclude something that we do not already know. Reasoning forms new beliefs from old rules via some rules or mechanisms. For example, if the browser window has not yet opened, you may reason that the application has not yet been launched, or that the tap was not registered. Another form of reasoning is *inference*, where we form a new belief based on observations. For example, we can infer that an icon in a different operating system, say macOS, will have the same function as the one we have seen in Windows.

Prediction is a special kind of reasoning that is needed to act effectively in a dynamic environment. Predictions are reasoning about the future. The challenge is to leap from observations about the present to what *will* or *might* happen in the future.

Reasoning is commonly needed when dealing with complex systems, for example:

- Reasoning about what a piece of software can do based on other knowledge about it, which may come from various sources such as advertisements, word of mouth, etc.
- Reasoning about what will happen if you issue a particular command to software you have not used before.
- Reasoning about the risks involved in a decision.

5.4.1 Mental models

Mental models are memory-based representations of interactive systems that are used for reasoning, inference, and prediction. They are representations of systems and the way users' inputs affect them. For example, a mental model of a thermostat may tell us how the temperature set by the user affects the perceived warmth in a room. By simulating a scenario in the mind using a mental model, the user can reason about the unobservable qualities of a system.

In a study of mental models, Mayer and Gallini [527] asked students to read descriptions of how a pump works. Some of these descriptions showed parts, operating steps, or both parts and steps, as shown in Figure 5.5. The parts-and-steps illustrations significantly helped the participants in both recall and problem-solving tasks. This finding indicates that mental representations of devices help users simulate what happens to the internal state of a device when a certain input is given.

The current understanding is that regular untrained users are often unable to form coherent and complete mental models of the devices they use. Mental models are rarely complete models. Instead, users' knowledge tends to be multifaceted and fragmented. For example, one may remember episodes of using a device and use this to recall how to operate it. One may also remember unrelated facts about what a device can do. In practical problem-solving situations, users are often reluctant to engage in effortful reasoning; they would rather just try out things and see what happens.

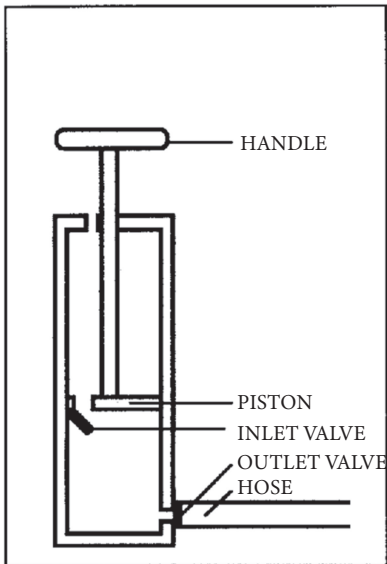
5.4.2 Decision-making

Decision-making refers to any situation in which multiple options are given and one or a subset must be chosen. Example: Your computer is infected with a virus. Which virus removal application would you pick?

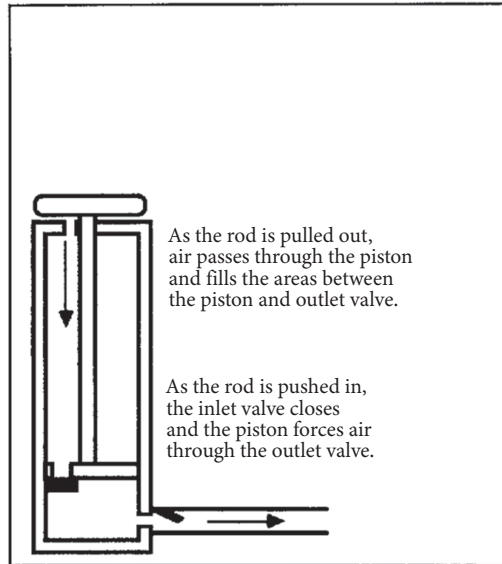
- Software A: Removes the virus with 95% probability but slows down the computer significantly in 5% of cases.
- Software B: Removes the virus with 70% probability but never slows down the computer.

In the more general case, N options are given, each associated with gain g (e.g., removing the virus) that occurs with probability p and a loss (e.g., slowing down the computer) that occurs with probability $1 - p$. Note that users may not be rational and always choose the option that is best for them.

PARTS ILLUSTRATION



STEPS ILLUSTRATION



PARTS AND STEPS ILLUSTRATION

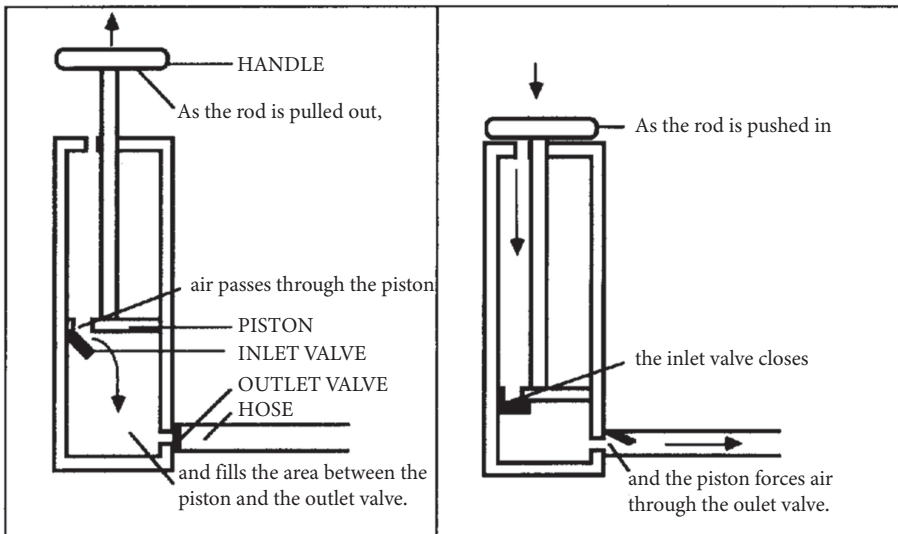


Figure 5.5 Mental models of a pump system used in the experiment of Mayer and Gallini [527].

Prospect theory posits that such choices are evaluated relative to a *reference point* [402]. For example, the status quo (the most common choice) or the most recent choice may work as a reference point. Such reference points guide how the user reasons about the gains and losses involved. The second claim from the theory is that some people are *risk-averse* about gains relative to the reference point, but risk-seeking about losses. In other words, they do not want to lose the gains provided by the reference point, but they are willing to gamble with losses. The third claim is that

some people are loss-averse. In other words, losing something hurts them more than gaining the same thing. For example, loss-averse gamers prefer an option that avoids the loss of a precious game award and view this as more important than the possibility of gaining the same award.

This theory is perhaps the most successful theory of economic decision-making, applied in studies of bargaining, consumer choice, voting, and even politics. In HCI, users face such decision-making all the time: How should we choose clothes for our avatar, our privacy settings, or the length of our passwords? These decisions involve various expected gains, losses, and risks. Many decisions in HCI can be analyzed through this lens; the challenge lies in estimating the gains and losses and their probabilities.

Multi-attribute choice refers to choice in the case where the options are characterized by multiple attributes. By contrast, in the previous example, there were only two attributes to consider: the efficiency of virus removal and the effect on the speed of the computer. We can generalize to more than two attributes. For example, if you want to buy a bicycle on an e-commerce site, you need to consider multiple attributes, such as price, type, size, location, and so on. Multi-attribute problems are hard for users, especially novices [494]. Their eye movement patterns are scattered; they can spend a lot of time going back and forth inspecting options, and they may feel unsatisfied with their eventual choice. Why is multi-attribute choice hard?

Users need to move their limited visual attention between rows to examine the options available. While doing that, they need to keep in mind the attributes they have considered so far, or at least those important for making a decision. They also need to reason about what the best option is and why. Few people can do this in a single pass since keeping all these attributes in mind is hard. Rather, what users often do is “satisficing”: They decide on what would be a good enough option and select it as soon as they find it. To circumvent the limits of WM, users may write down options, for example, in a spreadsheet, or seek external advice (reviews, opinions from friends). With experience, decisions become easier to make. Eye-movement strategies become less taxing and faster. Users learn to scan the options in a more systematic way and directly reject options that are irrelevant.

5.4.3 Decision heuristics

Kahneman [400] popularized the finding in neuroscience that we have two systems of decision-making. System 1 is a fast system driven by intuition, emotion, imagery, and associative memory. System 2 is a slow system that monitors System 1 and intervenes if intuition is not sufficient for the task. For example, you may feel like you want to click on a social media application to check the latest posts (System 1). However, as you start doing that, System 2 may intervene to stop that from happening, for example, because you have already checked that application recently.

In many circumstances, we do not have the possibility to evaluate all our options, and even if we did, the situation would be too complex for full analysis. In such cases, we use System 1. A *cognitive heuristic* is a rule of thumb used to identify a quick solution to a complex problem. For example:

Anchoring: It occurs when we center our choice around a known reference solution. For example, when choosing which application to use for some task, users often start with an application they already know.

Decoying: Decoying occurs when a reference point we have prevents us from seeing another one behind it. For example, a user who recalls ChatGPT as a large language model (LLM) that can answer questions may fail to remember the names of other LLMs.

Availability: Availability in heuristics refers to people's estimates of the probability of an event. Similar to its anchoring, the availability of a memory makes it more likely to be entered as a solution. PowerPoint, because of its prevalence as a slide editing software, may be the most likely option users consider even if there are many alternatives.

Status quo: Status quo bias refers to the prevailing solution, such as a famous or popular option. For example, popular products such as ChatGPT may first come to mind.

Bandwagon bias: Bandwagon bias occurs when we see our peers choosing the same option.

For example, if I ask you to name a great smartphone, you are likely to name something that comes to mind quickly, which may be the most recent smartphone model. This heuristic is called the *availability heuristic*.

The catch with heuristics is that while they allow us to generate a solution quickly, they limit the visibility of other solutions. They lead to *biases*. A bias, in this context, means a tendency to consider a skewed subsample in the space of options.

5.5 Simulating cognition in interactive tasks

Having described several aspects of cognition relevant to HCI, a natural question is how we would go about applying it to understand interactive tasks. *Cognitive models* are models that formally describe what happens in a person's mind during an interactive task. They provide an exact description of a cognitive mechanism—the way in which cognition mediates observable user behavior and the user's task.

A cognitive model receives some input that represents the task and has a mechanism to link that input to predictions about behavior. A model can be expressed in different ways:

Rule systems: Sets of rules or logical clauses that describe how information is processed in the mind. For example, mental models are described as logical statements.

Mathematical models: Statistical models that describe a relationship between factors related to the task or design, the outcomes of the interaction, and the cognitive factors that mediate the two.

Simulations: Stepwise executed computer programs. The best-known example is a class of cognitive architecture models, where different cognitive capacities, such as perception and memory, are program modules with their own inputs and outputs.

Data-driven models: Models developed using training data or via trial and error. For example, artificial neural networks can be used to understand aspects of visual saliency. Although such networks learn parameters from data, they also express architectural assumptions about the mind, such as the process of convolution.

In the early years of HCI, cognitive modeling had a central role in HCI [129]. One well-known example is Project Ernestine, which is discussed in detail in Paper Example 5.5.1. Another example is the goals, operators, methods, and selection rules (GOMS) model, an early cognitive model developed by Card et al. [129] that facilitated the development of cognitive architectures.

The development of cognitive architectures was inspired by computer architectures, which are descriptions of computer systems in terms of their underpinning components and their relationships. Cognition consists of serial and parallel information processing units. This terminology will be familiar to anyone who has studied computer science or software engineering:

Peripherals: These are sensors and actuators for interacting with the external environment. Examples include the oculomotor system, perception, and hearing.

Internal modules: Such modules may operate independently, processing inputs and sending outputs to other modules. They have capacity limitations, such as the number of items they can store or the amount of time they take to process input, and they can maintain internal states and run complex programs.

Production rules: These describe what is done and under which conditions. They are similar to computer programs that consist of a series of commands executed by modules.

Central processing unit: The central executive is limited in capacity and can only process things serially. When a program is run, it produces a *trace* of human behavior. Such a trace may contain the time needed to take an action or complete a task, the errors produced, and so on.

One of the main motivations for cognitive architectures in early HCI was that they could be used to decrease the cost of empirical evaluations with human participants. In addition, they were used to construct systems that adapt to the user, such as in cognitive tutoring, where cognitive models track a learner's cognitive development to select suitable study materials and interventions.

Why have cognitive models not been as widely adopted by practitioners as perhaps hoped at the outset of the field? First, applying a cognitive architecture model to an HCI problem requires a deep understanding of the user's task. Sometimes this understanding has to be so deep that it is easier to evaluate the design empirically than to build a model. For example, GOMS requires the researcher to specify rules that describe what the user does at the level of cognitive operations. Producing such descriptions can take weeks or even months. Approaches for lowering such barriers and helping practitioners to demonstrate tasks were not widely adopted, possibly because they still require significant work [388].

Second, cognitive models tend to work only within a narrow range of applications. A production system that has been created for one task and one particular design cannot be readily adopted for another setting—it must be updated and revised. In other words, models based on cognitive architecture have a scaling problem.

However, the topic of cognitive modeling is experiencing a revival with machine learning methods. New modeling approaches have been explored in HCI, most recently neural network models and reinforcement learning (Chapter 21). A key benefit of such approaches is that the researcher does not need to specify the production systems or policies. Instead, such models can learn them.

Paper Example 5.5.1: Project Ernestine

Project Ernestine is an example of cognitive modeling in HCI [289] that demonstrates the practical value of modeling cognition for HCI.

A telephone company was interested in improving the efficiency of their teleoperators. Teleoperators, a profession that no longer exists as switchboards are automated, were professionals who spoke with a caller and connected the call to its destination.

The company had designed a new workstation they believed would improve task performance and help the company save millions of dollars. They hired cognitive scientists to understand if the new design was better than the old one and, if so, by how much. The goal was simple: to estimate task completion times for the two workstation designs.

The researchers came back with a startling result: The new design was slower than the old one. The researchers pointed out that while the company had focused on the time that manual

operations take, the new design changed the cognitive requirements, with non-obvious consequences on performance. The new design required the recall of information, which is slow, and this factor was neglected by the company.

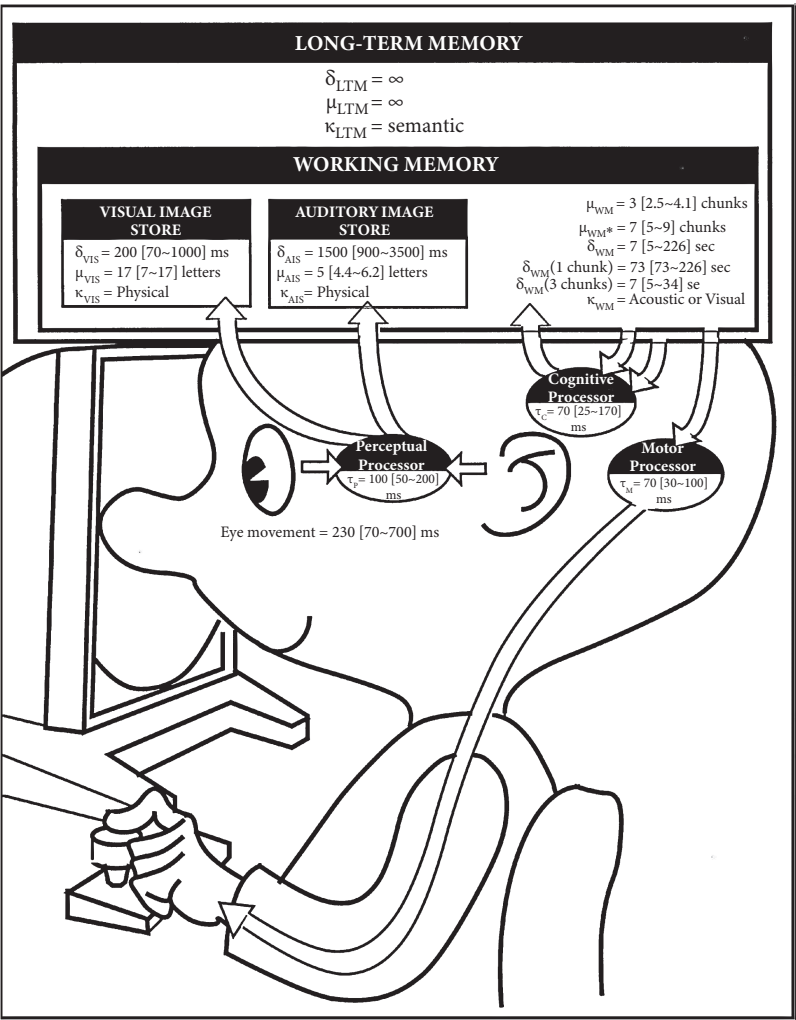


Figure 5.6 GOMS is a cognitive architecture model. It simulates the processing of information in separate modules of the mind and the flow of information between them. Modules have internal capacity limits and processing times [129].

To arrive at this conclusion, the authors used a variant of GOMS called CPM-GOMS. Figure 5.6 shows the cognitive architecture. CPM-GOMS predicted that the new workstation was 3% *slower*, not faster, than the old one. It further predicted that performance varied across categories of call-handling tasks. Thus, the researchers predicted the company would lose money if it switched to the new workstation.

continued

Paper Example 5.5.1: Project Ernestine (*continued*)

To ensure the results were trustworthy, the authors validated the results against empirical data they collected after the deployment of the new workstation. Comparing model predictions against real data on calls collected over four months, they learned that the average prediction error was small, varying between 11% and 12% in performance time, which corroborated their modeling approach.

Project Ernestine is an example of the special character of modeling in an applied field such as HCI. Modeling in HCI is often driven by practical needs in the evaluation and design of systems as opposed to a pursuit of knowledge for rigorous theory. Cognitive modeling can sometimes be used to evaluate designs and provide insights into their cognitive requirements.

Summary

- The main function of cognition in interaction is to help users deal with complex systems and situations.
- Cognition is limited, yet learning and adaptive.
- Theories of cognition help us answer several key questions about information-rich task environments, for example, what makes multitasking situations demanding, what happens when users learn to use a user interface, why they may fail to remember something they have seen before, and how they draw conclusions based on their beliefs about systems that are opaque to them.
- Many of HCI's guidelines and evaluative models are rooted in theories of cognition.

Exercises

1. Cognitive abilities. Measure your or your friend's cognitive abilities using PsyToolkit: <https://www.psytoolkit.org>. Carry out tests to assess WM capacity (e.g., n-back), cognitive control (e.g., task switching), and reasoning (e.g., Hanoi tower). Repeat each test a few times to obtain reliable measurements. What are some tasks in computer use where these abilities are important?
2. Cognitive load. Cognition requires energy and effort; how does design affect these? Take a computer game that you find mentally taxing. Ask a friend to play it and administer the NASA-TLX questionnaire after each level. Take measurements a few times for more robust estimates. Plot the components of the NASA-TLX as a function of the game level. What aspects of the game made each level less/more effortful?
3. Multiple resource theory. (a) Name two common tasks in mobile interaction that, according to MRT, would interfere with each other when carried out simultaneously, and illustrate the conflict by annotating the MRT cube given in Figure 5.2. (b) Explain why interference occurs. (c) Explain how either of the two tasks from (a) might be changed to reduce interference.
4. Observational study of human error. When was the last time you forgot your PIN code, missed your turn when driving, or forgot to unmute yourself on Zoom? Observational studies

are an important part of the HCI toolbox. To prepare for this task, read more on error taxonomies based on human cognition: <http://www.errordiary.org/blog/wp-content/uploads/HumanErrorSGSWMarch2013.pdf>. Fully understand the notions of activation, schema, and cross-talk. Then, download the error table Word file from the homepage of the book. It contains a table with eight error types on different rows: (a) capture, (b) double capture, (c) omission, (d) loss of activation, (e) description slip, (f) associative activation slip, (g) repetition of action slip, and (h) cross-talk slip. Your task is to observe an everyday interaction taking place with technology for one hour and to systematically identify errors using this taxonomy. We recommend a public interface similar to a vending machine. Ensure that (a) you can safely observe users, for example, by standing at a distance and taking auditory or written notes, and (b) the case is interesting from an error perspective, that is, different error types might appear.

5. Cognition and design. Pick a user interface design that you think is hard to use. Take full screenshots and analyze them using the design guidelines of Jeff Johnson listed in Table 41.1.
6. Memory. Human memory is important in interaction. Think about an interactive system that goes against the characteristics of memory discussed in this chapter, for example, a system that demands too much of our short-term memory. Propose how the design could be improved.